

Эссе: паттерны проектирования в Nix и NixOS

Дисциплина: Проектирование и архитектура программных систем **Тема:** Паттерны проектирования в выбранном языке программирования

0. Введение

Nix — это чисто функциональный, лениво вычисляемый предметно-ориентированный язык для декларативного описания программных пакетов и систем. Его создал Элко Долстра в 2003 году в рамках диссертации по воспроизводимым сборкам. На базе Nix-языка построены два больших проекта:

- **Nixpkgs** — крупнейший в мире репозиторий пакетов (>100 000 пакетов), целиком записанный на Nix;
- **NixOS** — Linux-дистрибутив, в котором вся система — от ядра и драйверов до systemd-юнитов и пользователей — описывается одним выражением Nix.

С точки зрения архитектуры программных систем интересно вот что: несмотря на то, что Nix — функциональный язык без классов, без наследования и без явного состояния, в Nixpkgs и NixOS отчётливо реализованы пять классических архитектурных паттернов из каталога GoF: **Builder**, **Template Method**, **Dependency Injection**, **Decorator** и **Composite**. Они приобретают неожиданно элегантную форму через функции высшего порядка и ленивые фиксированные точки.

Я расскажу о пяти таких паттернах в порядке от пакетного уровня к системному.

1. `stdenv.mkDerivation` — Builder + Template Method

Назначение. Стандартизировать сборку любого пакета: задать единый «скелет» процесса сборки (распаковать → пропатчить → сконфигурировать → собрать → установить), оставив подклассам право переопределить отдельные фазы.

Реализация в Nix. В Nixpkgs есть функция `stdenv.mkDerivation`, которая принимает атрибут-сет с описанием пакета и возвращает *derivation* — рецепт сборки. По умолчанию каждая фаза выполняется `bash`-скриптом, который можно заместить целиком или дополнить хуками `preBuild` / `postBuild`.

```
stdenv.mkDerivation rec {
  pname    = "hello";
  version  = "2.12.1";

  src = fetchurl {
    url      = "mirror://gnu/hello/hello-${version}.tar.gz";
    sha256   = "...";
  };

  # Никаких phase'ов не задано — будут использованы стандартные
  # unpackPhase / configurePhase / buildPhase / installPhase.

  # Хуки: запускаются между стандартными фазами.
  preBuild  = "echo собираем ${pname}";
  postBuild = ''
    echo собрали; cat -n Makefile | head
  '';
}
```

Какие паттерны GoF здесь работают. Сразу два:

- **Template Method.** `mkDerivation` определяет фиксированный алгоритм-каркас («скелет фаз»), а конкретные сборочные действия переопределяются в атрибут-сете. Это буквальное определение паттерна: «определяет скелет алгоритма в операции, передавая ответственность за некоторые его шаги подклассам».
- **Builder.** Сам атрибут-сет — fluent-описание сложного объекта (derivation) через именованные параметры: `src`, `buildInputs`, `nativeBuildInputs`, `configureFlags`, `installPhase` и т. д. Каждое поле — шаг строителя; результат — собранная derivation.

Где это используется. В нескольких десятках тысяч `default.nix`-файлов Nixpkgs. На уровне ОС это базовый кирпич: всё, что вы устанавливаете в NixOS, проходит через `mkDerivation`. Если бы в Nixpkgs не было этого шаблона, каждый пакет описывал бы пайплайн сборки заново.

2. `callPackage` — Dependency Injection / IoC

Назначение. Снять с разработчика пакета обязанность вручную «прокидывать» зависимости. Пакет объявляет, что ему нужно, через сигнатуру функции; контейнер `callPackage` сам найдёт нужные модули и передаст их.

Реализация в Nix. Каждый пакет в Nixpkgs — это **функция от своих зависимостей**, возвращающая derivation. А `callPackage` — это умная «фабрика», которая берёт такую функцию, смотрит на имена её аргументов и подставляет одноимённые атрибуты из `pkgs`:

```
# pkgs/applications/audio/myplayer/default.nix

{ stdenv, fetchurl, cmake, qt6, ffmpeg, alsa-lib }:

stdenv.mkDerivation {
  pname = "myplayer";
  src    = fetchurl { ... };
  nativeBuildInputs = [ cmake ];
  buildInputs      = [ qt6.qtbase ffmpeg alsa-lib ];
}
```

А вот так пакет регистрируется в общем дереве:

```
# pkgs/top-level/all-packages.nix

myplayer = callPackage ../applications/audio/myplayer { };
```

`callPackage` смотрит сигнатуру функции `{ stdenv, fetchurl, cmake, qt6, ffmpeg, alsa-lib }`, берёт из `pkgs` одноимённые поля и подставляет. Если в фигурных скобках после `callPackage ...` указать переопределение (`{ qt6 = pkgs.qt5; }`), будут использованы они вместо дефолтных. Это и есть **внедрение зависимостей**: разработчик пакета не знает, откуда возьмётся `qt6` — её даёт контейнер.

Какой паттерн GoF. Чистая **Dependency Injection** через сигнатуру конструктора, реализованная как IoC-контейнер. По форме это ещё **Factory Method**: `callPackage` — фабрика, которая «знает», как создать пакет из имени.

Что это даёт. Подменить любую зависимость целого подграфа можно одной строкой в overlay'e, без правки самих пакетов. Например, `pkgs.callPackage path { openssl = openssl_3; }` пересоберёт пакет на другой версии OpenSSL — это лежит в основе всех runtime-вариантов в NixOS.

3. `lib.fix` — Fixed-Point Recursion / Late Binding

Назначение. Позволить объекту ссылаться на «самого себя в финальном виде» — то есть на свою конечную, уже расширенную всеми надстройками версию. В ООП это называется *late binding* (позднее связывание) и обеспечивается виртуальными методами.

Реализация в Nix. В стандартной библиотеке Nixpkgs есть простая, но магически работающая функция:

```
fix = f: let self = f self; in self;
```

Поскольку Nix ленив, `self` тут — это рекурсивная ссылка, которая разворачивается только при обращении к конкретным полям. Этот трюк позволяет писать:

```

let
  packagesGenerator = self: {
    a = 1;
    b = 2;
    c = self.a + self.b;      # ссылка на «финального себя»
    d = self.c * self.c;
  };

  pkgs = lib.fix packagesGenerator;
in
  pkgs.d                      # = 9

```

Здесь `c` и `d` могут ссылаться на другие поля того же атрибут-сета через `self`. Но самое интересное — что любую функцию-генератор можно «обернуть»:

```

lib.fix (self:
  let base = packagesGenerator self;      # обычные пакеты
  in base // { a = 100; })                # с подменой a на 100
# теперь c = 102, d = 10404

```

То есть мы перевычисляем граф пакетов с учётом подмены, и `d`, которое зависит от `c`, которое зависит от `a` — автоматически пересчитается.

Какой паттерн GoF. Это форма **Dependency Injection**, доведённая до системного уровня: каждый пакет «знает» о других через `self`, но не зависит от конкретной реализации этих других. Это же родительский механизм для следующего паттерна — *Overlay*.

Где применяется. Корень всего Nixpkgs — это `lib.fix` от большого «генератора пакетов». Когда вы делаете `pkgs.firefox`, фактически вы достаёте поле из фиксированной точки гигантской функции, в которой каждое поле может ссылаться на каждое другое.

4. Overlay — Decorator + Open/Closed

Назначение. Расширять или модифицировать Nixpkgs локально, не форкая репозиторий — это аналог open/closed principle: «открыт для расширения, закрыт для модификации».

Реализация в Nix. Overlay — это функция двух аргументов:

```
overlay = final: prev: {
  myhello = prev.hello.overrideAttrs (old: {
    patches = (old.patches or []) ++ [ ./my-fix.patch ];
  });

  shiny = final.callPackage ./shiny { };      # видит overlay'и
  legacy-shiny = prev.callPackage ./shiny { }; # видит pkgs без overlay'ев
};
```

- `prev` — слой `pkgs` до этого overlay'я (всё, что было определено раньше);
- `final` — финальная фиксированная точка, в которой все overlay'и уже применены.

Внутри `Nixpkgs` `final` и `prev` производятся через тот самый `lib.fix`: цепочка overlay'ев — это последовательные «декораторы», каждый из которых получает прежний слой и финальную проекцию, и возвращает свои изменения. Финальная фиксированная точка вычисляется уже после всех overlay'ев.

```
# В flake.nix:
pkgs = import nixpkgs {
  inherit system;
  overlays = [ myOverlay anotherOverlay ];
};
```

Какие паттерны GoF. Очень чистый **Decorator**: overlay оборачивает существующий граф пакетов, добавляя/изменяя поведение без изменения исходного кода. Поскольку overlay'и komponуются стопкой, это ещё и пример **Chain of Responsibility** на уровне сборки графа: каждый overlay видит результат предыдущих и может его дополнить.

Практический смысл. Если мне нужна собственная пропатченная версия `firefox`, достаточно положить overlay рядом с конфигом — оригинальный `nixpkgs` остаётся неизменным, обновление через `nix flake update` не сломает локальные правки.

5. NixOS Module System — Composite + Strategy

Назначение. Описывать целую систему — Linux-дистрибутив со всеми сервисами, пользователями, ядром, GUI — как композицию независимых самодостаточных модулей.

Реализация в Nix. Модуль NixOS — это атрибут-сет с тремя разделами:

```
# /etc/nixos/modules/games.nix
{ config, lib, pkgs, ... }:
{
  options.programs.games = {
    enable = lib.mkEnableOption "набор игр";
    extra = lib.mkOption {
      type = lib.types.listOf lib.types.package;
      default = [];
    };
  };

  config = lib.mkIf config.programs.games.enable {
    environment.systemPackages =
      [ pkgs.steam pkgs.lutris ] ++ config.programs.games.extra;
  };
}
```

- **options** — какие настройки этот модуль *публикует*. Это типизированные поля (`types.bool`, `types.listOf`, `types.submodule`), у каждого есть тип, default, mergeStrategy и описание.
- **config** — что этот модуль *вкладывает* в общесистемную конфигурацию (которая тоже задаётся через те же options, но определённые в других модулях).
- **imports** — список других модулей, на которые этот опирается.

NixOS-система собирается так: указываются все модули (`/etc/nixos/configuration.nix` импортирует десятки), их **options** объединяются в единую схему, каждый **config** вкладывает свою часть в общесистемный атрибут. **Merge-стратегии разрешают конфликты**: списки конкатенируются, словари объединяются, boolean'ы объединяются через OR, и т. д.

Какие паттерны GoF. Сразу два, в красивой связке:

- **Composite.** Вся NixOS-система — это «дерево», корень которого — итоговый **config**, а листья — отдельные модули. Каждый модуль, в свою очередь, может импортировать ещё модули — иерархия рекурсивна. Внешне обращение к `config.networking.firewall.allowedTCPPorts` идёт через единый интерфейс, не различая, в каком модуле этот option объявлен и в скольких модулях его дополнили.
- **Strategy.** Каждый отдельный модуль — самостоятельная «стратегия» того, как реализовать какой-то аспект системы. Например, `nix-modules/services/networking/firewall-iptables.nix` и `firewall-nftables.nix` обе предоставляют один и тот же option `networking.firewall`, но реализуют его через разные backends. Поменять backend — поменять один импорт.

Связанный паттерн — submodule. Тип `types.submodule { options = {...}; }` позволяет вкладывать модули внутрь option'ов, превращая дерево конфигурации в полноценное рекурсивное дерево объектов (например, `services.nginx.virtualHosts.<name>` — это атрибут, чьи значения сами по себе модули с собственными options).

6. Заключение

Nix демонстрирует, что архитектурные паттерны проектирования — это не свойство языка с классами, а свойство **сложных программных систем как таковых**. В Nix роль классов играют функции, роль наследования — overlays и `lib.fix`, роль интерфейсов — сигнатуры функций и типы option'ов, но решаемые задачи остаются прежними: декомпозиция, повторное использование, замещаемость, расширение без модификации.

Пять рассмотренных паттернов работают в Nix как единая система:

```
mkDerivation (Builder + Template Method)
    ↑
    | создаёт пакет
    |
callPackage (DI / IoC) — собирает дерево пакетов с подстановкой зависимостей
    ↑
    | строит pkgs как fix(self -> ...)
    |
lib.fix (Late binding) — позволяет ссылаться на «себя финального»
    ↑
    | применяется к стопке overlay'ев
    |
Overlay (Decorator + Chain of Responsibility) — модифицирует pkgs неинвазивно
    ↑
    | pkgs передаётся в NixOS-модули
    |
NixOS modules (Composite + Strategy) — собирают систему из независимых модулей
```

Эта вертикаль — от единичной derivation до целой ОС — устроена на одних и тех же абстракциях. По сути перед нами доказательство, что хорошие паттерны проектирования универсальны и переносимы в чисто функциональную парадигму: им не нужны классы, им нужна композиция.

Приложение: список GoF-паттернов, упомянутых в эссе

Паттерн GoF	Где встречается в Nix
Builder	<code>stdenv.mkDerivation</code> — атрибут-сет как «строитель» derivation
Template Method	<code>mkDerivation</code> -фазы (unpack/configure/build/install) с возможностью переопределения
Factory Method	<code>callPackage</code> как фабрика, выбирающая аргументы по имени
Dependency Injection / IoC	<code>callPackage</code> + <code>lib.fix</code>
Decorator	overlay над <code>pkgs</code>
Chain of Responsibility	цепочка overlay'ев, каждый получает prev/final
Composite	NixOS-модули, submodule, иерархия option'ов
Strategy	альтернативные модули, реализующие одни и те же options

Литература и источники для дальнейшего чтения

1. Eelco Dolstra. *The Purely Functional Software Deployment Model*. PhD thesis, Utrecht University, 2006.
2. *NixOS manual — Writing NixOS modules*, разделы «Options» и «Imports».
3. *Nixpkgs manual — Overlays*.
4. Gamma, Helm, Johnson, Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.